

CPU Scheduling Algorithms Comparison & Analysis

Round Robin · Shortest Job First · SRTF

Operating Systems Project Report

Technical Explanation & Performance Analysis

Subject	Operating Systems
Topic	CPU Process Scheduling
Algorithms	Round Robin · Non-Preemptive SJF · Preemptive SJF (SRTF)

-
1. Introduction
 2. Project Objectives
 3. Algorithms Overview
 - 3.1 Round Robin (RR)
 - 3.2 Non-Preemptive SJF
 - 3.3 Preemptive SJF — SRTF
 4. System Features
 5. Technical Explanation
 - 5.1 System Architecture
 - 5.2 Input Layer & Validation
 - 5.3 Scheduling Engine
 - 5.4 Gantt Chart Visualization
 - 5.5 Metrics Calculation
 6. Performance Analysis
 7. Analysis Questions
 8. Conclusion
 9. System Screenshots

1. Introduction

CPU scheduling is one of the most critical responsibilities of an operating system. The scheduler determines which process receives CPU time at each moment, directly influencing system throughput, responsiveness, and resource utilisation efficiency.

Different scheduling algorithms are designed with different goals. Some focus on fairness and responsiveness; others optimise efficiency and minimise waiting time. This project implements and compares three representative algorithms:

- Round Robin (RR) — fairness-oriented, preemptive time-sharing.
 - Non-Preemptive Shortest Job First (SJF) — efficiency-oriented.
 - Preemptive Shortest Job First / SRTF — aggressive optimisation.
-

2. Project Objectives

- Implement multiple CPU scheduling algorithms correctly.
 - Compare algorithm performance using identical workloads.
 - Visualise execution order using Gantt Charts.
 - Calculate and analyse scheduling performance metrics.
 - Study the effect of preemption on scheduling behaviour.
 - Handle invalid user input safely through comprehensive validation.
-

3. Algorithms Overview

3.1 Round Robin (RR)

Round Robin is a preemptive scheduling algorithm designed for fairness and time-sharing systems. Each process receives a fixed CPU execution slice called the **Time Quantum**. If a process does not complete within its quantum, it is moved to the end of the ready queue and the next process is served. The scheduler rotates processes in a circular queue until all processes finish execution.

Advantages	Disadvantages
■ Fair CPU distribution among all processes	■ Large number of context switches
■ Fast first response time	■ Performance highly depends on quantum size
■ Suitable for interactive and multitasking systems	■ Large quantum resembles FCFS behaviour
■ Prevents CPU monopolisation	■ Small quantum increases overhead

Scheduling Logic

Step	Action
1	Add all arrived processes to the ready queue.
2	Select the first process in the queue.
3	Execute the process for one quantum (reduce remaining burst time).
4	If unfinished, move the process to the end of the queue.
5	Repeat until all processes complete.

3.2 Non-Preemptive Shortest Job First (SJF)

Shortest Job First selects the process with the smallest burst time among all currently available ready processes. Once execution starts, the selected process runs until completion without interruption. SJF focuses primarily on minimising average waiting time and turnaround time.

Advantages	Disadvantages
■ Very low average waiting time	■ Long processes may suffer starvation
■ High efficiency for short jobs	■ Poor fairness across processes
■ Low turnaround time	■ Requires burst time estimation
■ Simple scheduling decision	■ CPU is never interrupted once started

Scheduling Logic

Step	Action
1	Check all arrived processes.

2	Select the process with the shortest burst time.
3	Execute it until completion (no preemption).
4	Repeat until all processes complete.

3.3 Preemptive SJF — SRTF

Shortest Remaining Time First (SRTF) is the preemptive extension of SJF. Instead of selecting the shortest burst only at dispatch time, the scheduler continuously compares remaining execution times. If a newly arrived process has a shorter remaining burst than the currently running process, the CPU immediately switches to the new process. This aggressive optimisation significantly improves average waiting time for short processes.

Advantages	Disadvantages
■ Lower waiting time than non-preemptive SJF	■ High context-switching overhead
■ Better turnaround time optimisation	■ Long processes may experience starvation
■ Very efficient for short processes	■ More complex implementation
■ Dynamically adapts to newly arriving jobs	■ Frequent interruptions reduce fairness

Scheduling Logic

Step	Action
1	Continuously monitor arriving processes.
2	Compare remaining burst times of all ready processes.
3	Execute the process with the shortest remaining time.
4	If a shorter process arrives, interrupt the current process.
5	Switch execution to the shorter process immediately.
6	Continue until all processes finish execution.

4. System Features

- Interactive graphical user interface (GUI)
- Dynamic process management — add and remove processes at runtime
- Configurable Time Quantum for Round Robin
- Scheduling visualisation using Gantt Charts
- Automatic metrics calculation (CT, TAT, WT, RT)
- Average metrics comparison across all algorithms
- Comprehensive input validation system
- Quick test-case generation for rapid evaluation
- Side-by-side comparison summary section
- Error handling with descriptive messages for invalid inputs

5. Technical Explanation

5.1 System Architecture

The project is structured into four independent but interconnected layers, ensuring a clean separation of concerns:

Layer	Responsibility
Input Layer	Accepts Process ID, Arrival Time, Burst Time, Time Quantum; validates all fields before execution.
Scheduling Engine	Executes all three algorithms independently on the same workload; tracks current time, remaining burst, queue updates, and completion status.
Metrics Calculation	Computes CT, TAT, WT, RT per process, then derives per-algorithm averages.
Visualization & Comparison	Renders Gantt Charts (start/end times, preemption events) and a side-by-side comparison summary.

5.2 Input Layer & Validation System

The input layer collects Process ID, Arrival Time, Burst Time, and Time Quantum. The validation system rejects the following to prevent undefined scheduling behaviour:

- Negative arrival times
- Zero or negative burst times
- Invalid or non-positive quantum values
- Duplicate process IDs

- Missing or empty input fields

Error messages are displayed automatically whenever invalid data is detected, ensuring robust and safe operation.

5.3 Scheduling Engine

The scheduling engine executes all three algorithms independently using the same workload to guarantee a fair performance comparison. It tracks the following state variables throughout execution:

- Current simulation time
- Remaining burst time per process
- Process states (waiting, running, completed)
- Ready queue updates
- Completion status flags

5.4 Gantt Chart Visualisation

The simulator generates Gantt Charts to represent the CPU execution timeline visually. Each chart displays:

- Process execution sequence in time order
- Start and end timestamps per time slice
- Preemption events (SRTF) and queue rotation events (RR)
- Idle CPU periods (if any)

5.5 Metrics Calculation

The system computes the following scheduling metrics for every process and derives algorithm-level averages for comparison:

Metric	Formula	Description
Completion Time (CT)	—	Moment when a process finishes execution completely.
Turnaround Time (TAT)	CT – AT	Total time in system from arrival to completion.
Waiting Time (WT)	TAT – BT	Time spent waiting inside the ready queue.
Response Time (RT)	FirstStart – AT	Delay before first CPU execution. Critical for interactive systems.
Avg. Waiting Time	ΣWT / n	Overall scheduling efficiency indicator.
Avg. Turnaround Time	ΣTAT / n	Average speed of process completion.
Avg. Response Time	ΣRT / n	Overall system responsiveness for users.

Lower average values indicate better scheduling efficiency and user experience.

6. Performance Analysis

The project results demonstrated significant behavioural differences between the implemented algorithms when evaluated on identical process workloads:

- **SRTF** achieved the lowest average waiting and turnaround times by continuously interrupting longer processes whenever a shorter job arrived.
- **Round Robin** achieved the best average response time, ensuring every process received CPU access quickly through periodic time-slicing.
- **Non-Preemptive SJF** significantly reduced waiting time for short jobs compared to Round Robin, without the implementation complexity of SRTF.
- Long processes experienced greater waiting time under SJF and SRTF due to prioritisation of shorter jobs, raising starvation concerns.
- Round Robin's performance is sensitive to the chosen quantum: smaller values improve responsiveness but increase context-switch overhead; larger values reduce overhead but approach FCFS behaviour.

Algorithm Comparison Summary

Criterion	Round Robin	SJF (Non-Preemptive)	SRTF (Preemptive)
Avg. Waiting Time	High	Low	Lowest
Avg. Response Time	Best	Poor	Moderate
Fairness	Excellent	Poor	Poor
Starvation Risk	None	High	High
Context Switches	Many	Few	Many
Complexity	Low	Low	High
Best Use Case	Interactive / Time-sharing	Batch / Efficiency-focused	Short-job-heavy workloads

7. Analysis Questions

Q1: Which algorithm produced the lowest average waiting time?

Preemptive SJF (SRTF) produced the lowest average waiting time. By continuously prioritising the process with the shortest remaining burst time — even interrupting longer running processes — SRTF minimises the time any short process spends waiting in the queue.

Q2: Which algorithm produced the lowest average response time?

Round Robin produced the lowest average response time. Because every process receives CPU execution periodically through time-slicing, no process waits long before its first CPU assignment, making RR ideal for interactive systems.

Q3: Did priority values improve treatment of urgent processes?

Priority scheduling was not implemented directly in this project. However, the analysis shows that policies which implicitly prioritise shorter or more urgent tasks (such as SJF and SRTF) do improve their treatment at the cost of fairness to longer processes.

Q4: Did SRTF favour short jobs more aggressively than SJF?

Yes. SRTF strongly favours short jobs by immediately interrupting longer processes whenever a process with a shorter remaining burst arrives. This dynamic preemption significantly reduces waiting and turnaround times for short processes, but increases context-switching overhead and starvation risk for long processes.

Q5: Which algorithm would you recommend for your tested workload, and why?

The best choice depends on system requirements: Round Robin is recommended for interactive and time-sharing systems where fairness and fast response matter. Non-Preemptive SJF suits batch or efficiency-focused environments with predictable job lengths. SRTF delivers maximum efficiency when short processes dominate the workload and starvation risk is manageable. No single algorithm is universally optimal — the decision must reflect workload characteristics and system goals.

8. Conclusion

This project successfully implemented and compared three fundamental CPU scheduling algorithms — Round Robin, Non-Preemptive SJF, and Preemptive SJF (SRTF) — using a unified simulator that produces Gantt Charts and quantitative performance metrics.

Algorithm	Key Strength	Primary Trade-off
Round Robin	Fairness · Fast response time · No starvation	High context switches; quantum-sensitive performance
SJF (Non-Preemptive)	Low average waiting time · Simple logic	Starvation of long jobs; no preemption
SRTF (Preemptive)	Highest efficiency · Optimal short-job handling	High overhead; starvation risk; complex to implement

The experiment confirmed that no single scheduling algorithm is universally optimal. The right choice depends on system requirements: interactive systems benefit from Round Robin's fairness, efficiency-oriented batch systems from SJF's low waiting times, and short-job-dominated workloads from SRTF's aggressive optimisation. Understanding these trade-offs is essential for designing responsive and efficient operating system schedulers.

9. System Screenshots

The following screenshots illustrate the simulator's graphical interface, Gantt Chart outputs, per-process metric tables, and the input validation system across all tested scenarios.

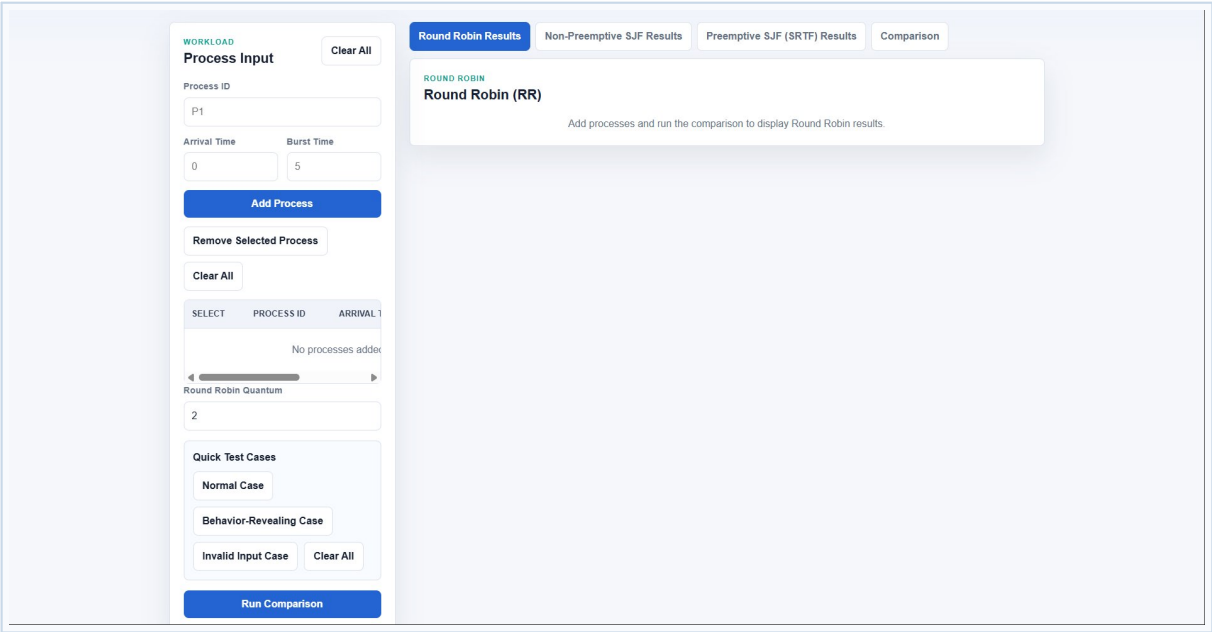


Figure 1 — Home Interface: Process Input panel with Quick Test Cases and Round Robin Quantum configuration.

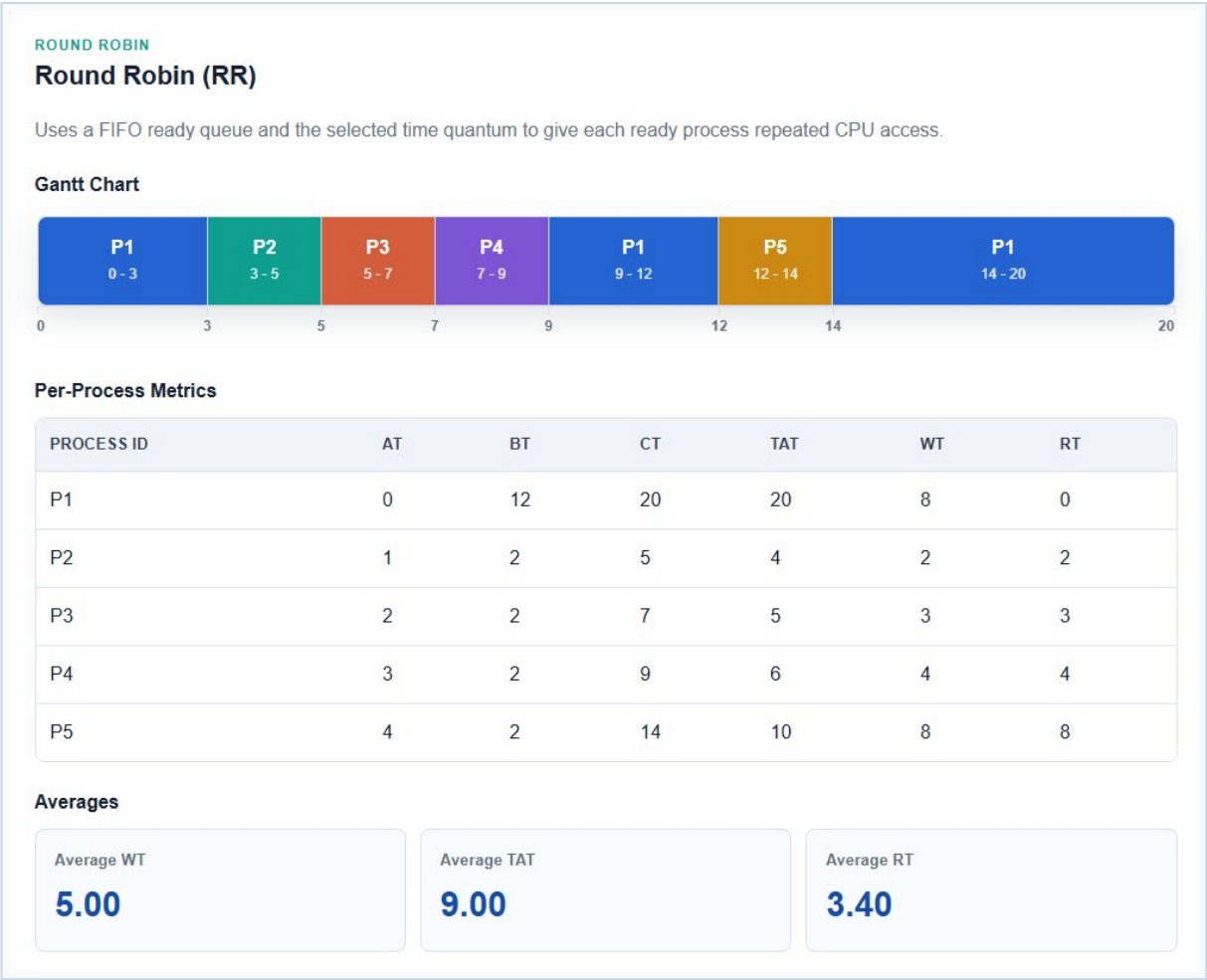


Figure 2 — Round Robin Results: Gantt Chart and per-process metrics (Avg WT: 5.00 · Avg TAT: 9.00 · Avg RT: 3.40).

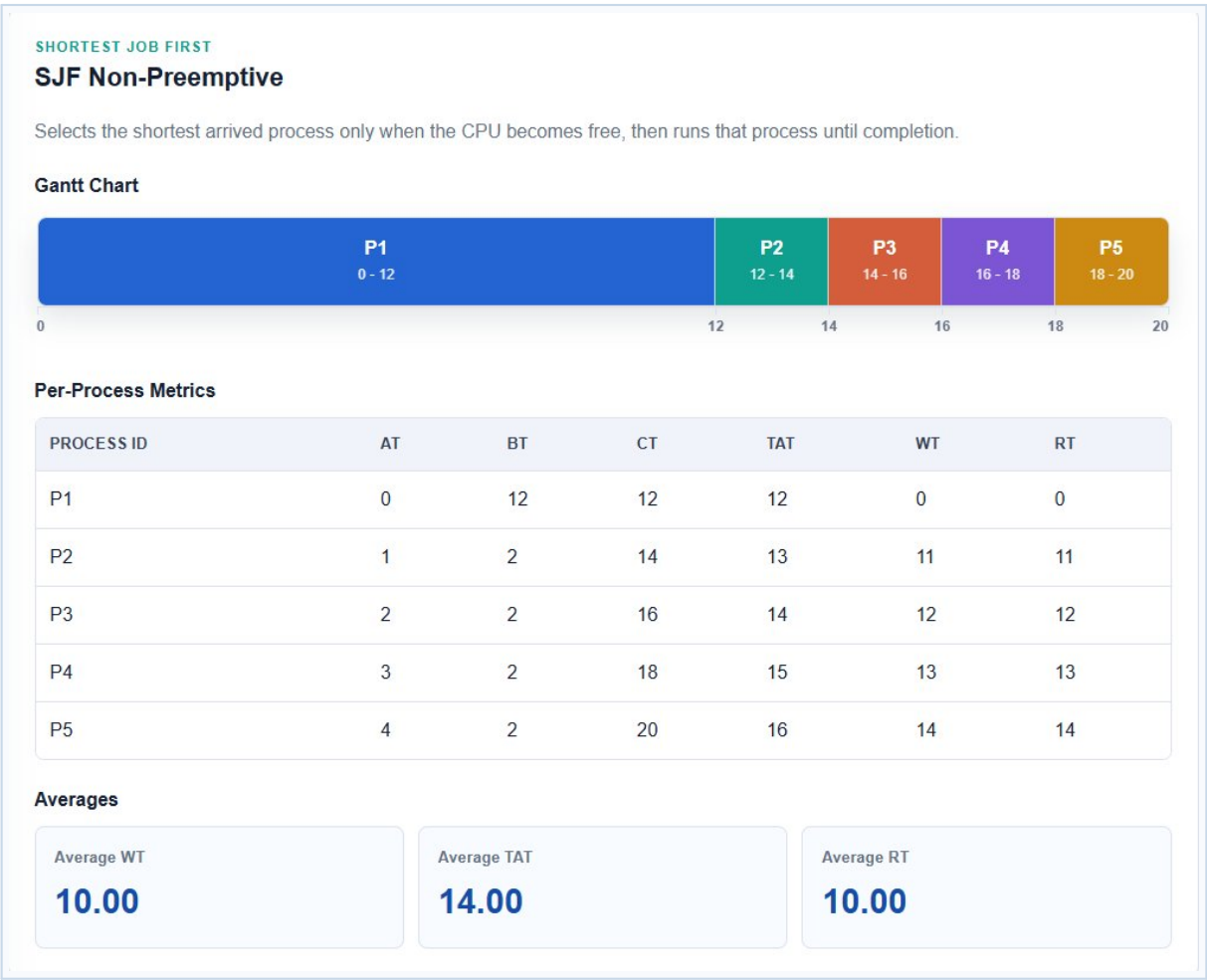


Figure 3 — SJF Non-Preemptive Results: Gantt Chart showing P1 monopolising CPU until t=12 (Avg WT: 10.00 · Avg TAT: 14.00 · Avg RT: 10.00).

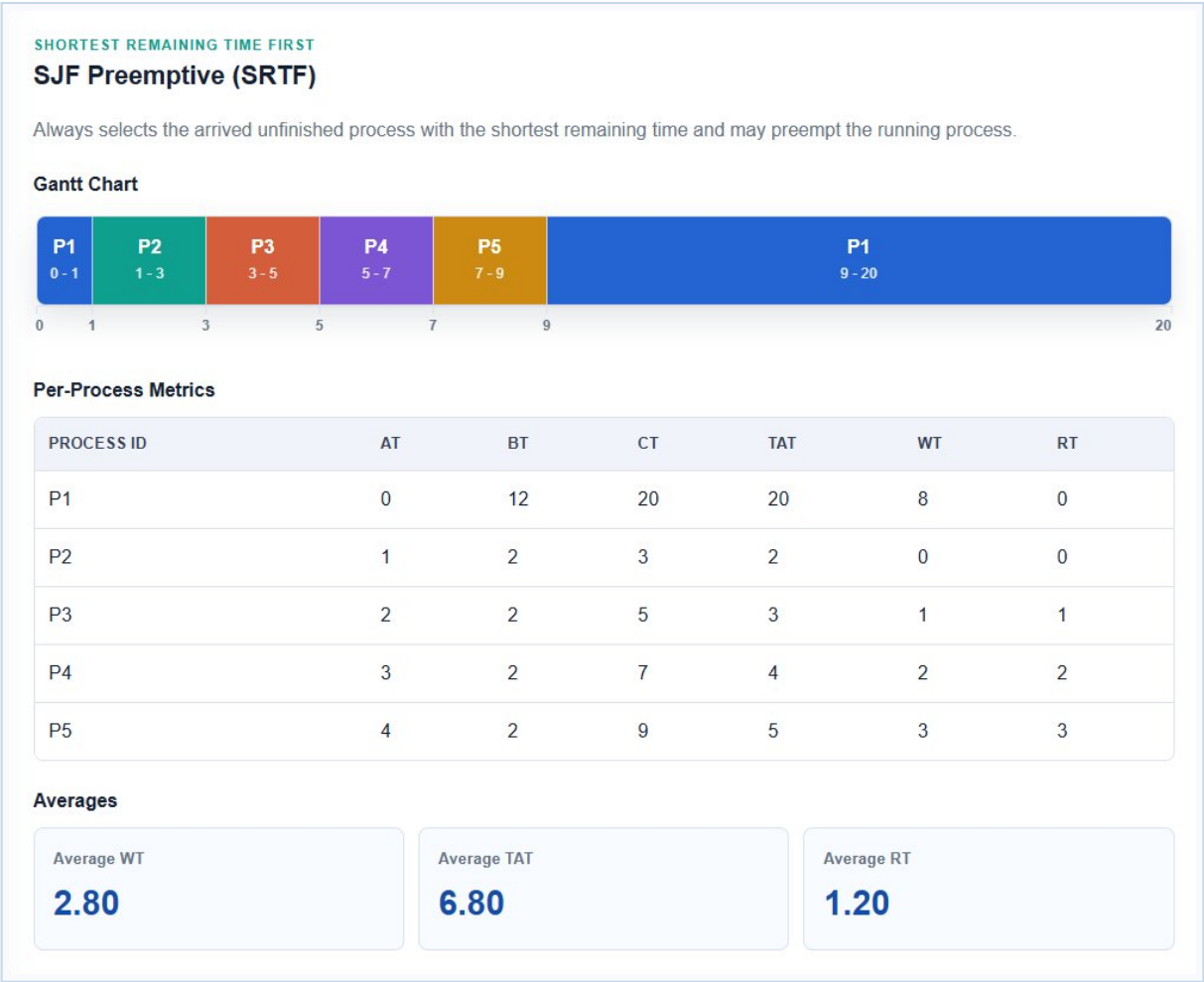


Figure 4 — SJF Preemptive (SRTF) Results: Gantt Chart with aggressive preemption giving best efficiency (Avg WT: 2.80 · Avg TAT: 6.80 · Avg RT: 1.20).

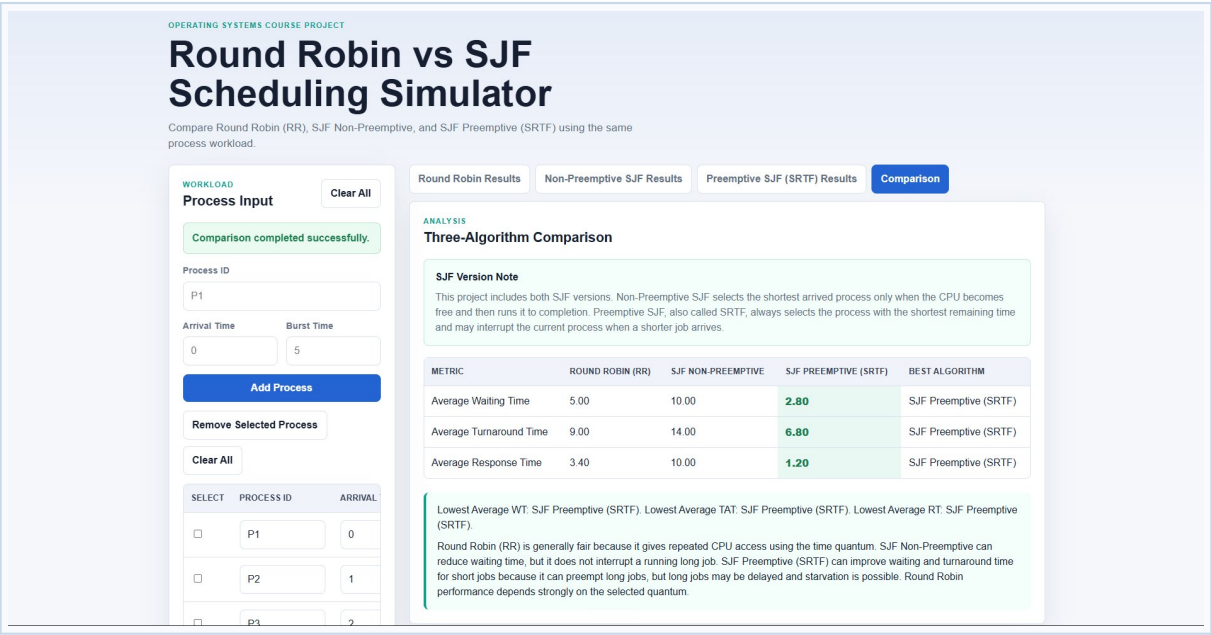


Figure 5 — Three-Algorithm Comparison: Side-by-side metric table highlighting SRTF as the best performer across all average metrics.

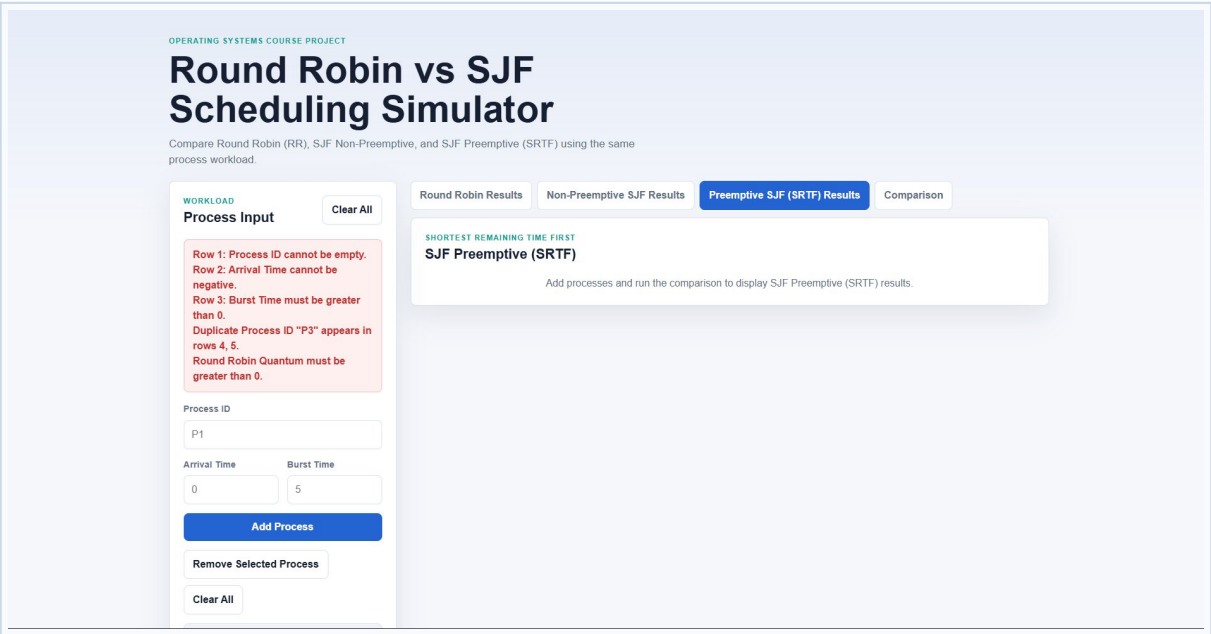


Figure 6 — Input Validation: Error messages displayed for empty Process ID, negative Arrival Time, zero Burst Time, duplicate IDs, and invalid Quantum.